

LAB-4: Dependency Isolation (All Variants)

Goal: Extend LAB-3 so that the selected strategy is isolated from concrete implementations and allows controlling dependency behavior using Fake and Stub.

General Requirements

- Program.cs works only with interfaces - No business logic in Main method - No 'new' inside classes (use constructor injection) - Fake and Stub implementations are created - At least two logical branches demonstrated - README explains what was isolated and why

VARIANT 1 – Student Printing Strategy (IStudentPrinter)

Situation:

The printing mechanism may change in the future (e.g., from Console to File or API). You need to isolate presentation logic from business logic.

```
public interface IStudentPrinter
{
    void Print(Group group);
}
```

Requirements: 1. Isolate this dependency via constructor injection. 2. Create a Fake class (stable result). 3. Create a Stub class (controllable result). 4. Demonstrate at least two business logic scenarios. 5. Explain in README what problem was solved.

VARIANT 2 – Average Calculation Strategy (IAverageStrategy)

Situation:

Average calculation rules may change (Simple, Weighted, etc.). You need to isolate the calculation mechanism from StudentService.

```
public interface IAverageStrategy
{
    double Calculate(Student student);
}
```

Requirements: 1. Isolate this dependency via constructor injection. 2. Create a Fake class (stable result). 3. Create a Stub class (controllable result). 4. Demonstrate at least two business logic scenarios. 5. Explain in README what problem was solved.

VARIANT 3 – Menu Abstraction (IMenuService)

Situation:

The menu mechanism may change (Basic, Advanced, Web UI, etc.). You need to isolate user interaction from the core logic.

```
public interface IMenuService
{
    void Run();
}
```

Requirements: 1. Isolate this dependency via constructor injection. 2. Create a Fake class (stable result). 3. Create a Stub class (controllable result). 4. Demonstrate at least two business logic scenarios. 5. Explain in README what problem was solved.

VARIANT 4 – Search Strategy (IStudentFinder)

Situation:

Search may be performed by ID, Email, or external API. You need to isolate the search mechanism from business logic.

```
public interface IStudentFinder
{
    Student? Find(Group group, string query);
}
```

Requirements: 1. Isolate this dependency via constructor injection. 2. Create a Fake class (stable result). 3. Create a Stub class (controllable result). 4. Demonstrate at least two business logic scenarios. 5. Explain in README what problem was solved.

VARIANT 5 – Validation (IStudentValidator)

Situation:

Validation rules may be strict or flexible. You need to isolate validation rules from StudentService.

```
public interface IStudentValidator
{
    bool Validate(Student student);
}
```

Requirements: 1. Isolate this dependency via constructor injection. 2. Create a Fake class (stable result). 3. Create a Stub class (controllable result). 4. Demonstrate at least two business logic scenarios. 5. Explain in README what problem was solved.

Evaluation (10 points)

2 points – Correct dependency isolation 2 points – Fake implementation 2 points – Stub implementation 2 points – Clear demonstration of logical branches 2 points – README with architectural explanation